

Network Protocols

Complete Study Notes

OSI Model · Client-Server · P2P · TCP · UDP
HTTP · HTTPS · FTP · SMTP · IMAP · POP

TOPICS COVERED

1. Network Protocols & OSI Model

2. Client-Server Model

3. Peer-to-Peer (P2P) Model

4. HTTP · HTTPS · FTP · SMTP

5. IMAP vs POP — Email Stack

5. TCP — Three-Way Handshake

6. UDP — Fire and Forget

7. Protocol Selection Framework

8. Security & Quick Revision

9. Interview Questions

Table of Contents

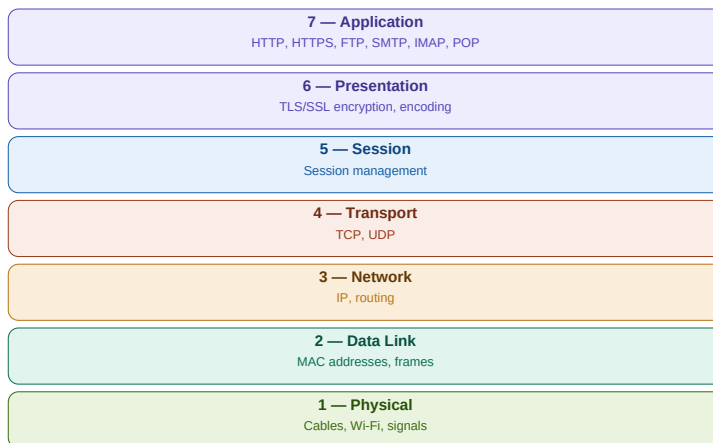
1. What Are Network Protocols?	3
2. OSI Model — Protocol Layers	3
3. Communication Models	3
3.1 Client-Server Model	3
3.2 Peer-to-Peer (P2P) Model	4
4. Application Layer Protocols	4
4.1 HTTP 4.2 HTTPS 4.3 FTP	4
4.4 SMTP 4.5 IMAP vs POP	5
5. Transport Layer — TCP & UDP	6
5.1 TCP · 5.2 UDP · 5.3 Comparison	6
6. Protocol Selection by Application	8
7. Security — HTTP vs HTTPS	8
8. Quick Revision Points	9
9. Interview Questions & Key Terms	9

1. What Are Network Protocols?

A **network protocol** is a set of rules defining how two devices communicate — covering message format, sequencing, error detection, and connection management. Both devices must use the same protocol to understand each other.

2. OSI Model — Protocol Layers

OSI Model — Protocol Layers



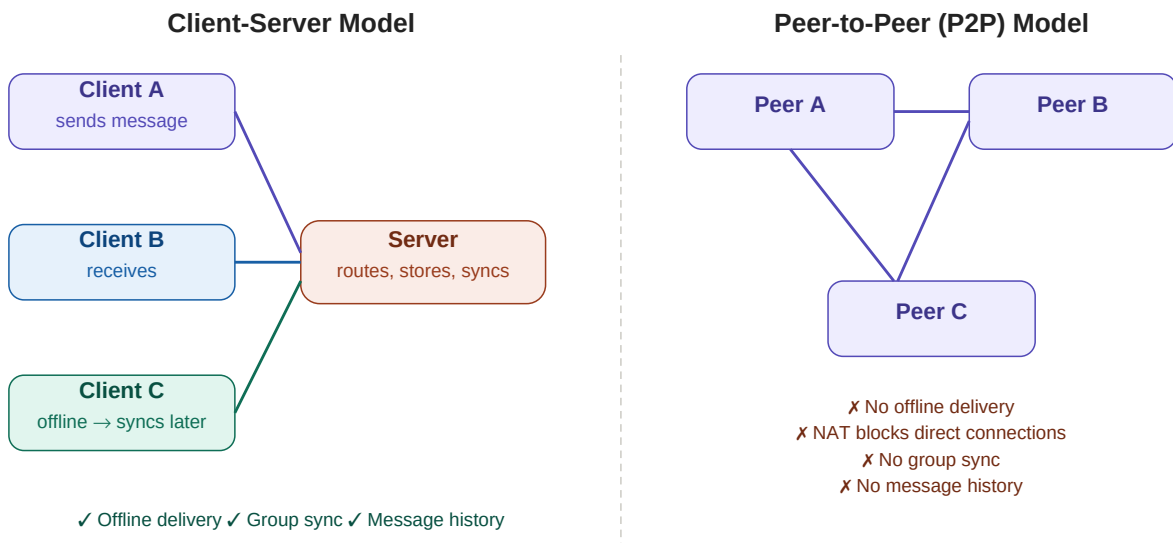
OSI Model — Layer 4 (TCP/UDP) and Layer 7 (HTTP/SMTP/FTP) are the main focus

Layer 4 — Transport: TCP and UDP. Port numbers, segmentation, reliability.

Layer 7 — Application: HTTP, HTTPS, FTP, SMTP, IMAP, POP.

Interview: State the layer when naming a protocol. TCP = L4. HTTP = L7.

3. Communication Models



Client-Server (left) vs Peer-to-Peer (right)

3.1 Client-Server

A central server stores, routes, and delivers messages. Clients initiate; server responds. Standard for messaging apps, email, and the web.

Advantages	Disadvantages
Reliable delivery & offline storage	Server infrastructure cost
Multi-device sync, group messaging	Potential single point of failure
Horizontally scalable	Higher latency than direct peers

3.2 Peer-to-Peer (P2P)

Clients connect directly — no server. Works for file sharing (BitTorrent) but fails for messaging due to:

- **No offline storage** — message lost if recipient is offline
- **NAT blocking** — home routers prevent direct peer connections
- **No group sync** — $O(N^2)$ complexity with N peers

WhatsApp is NOT P2P — it uses client-server. Websites are NOT P2P — HTTP is client-server.

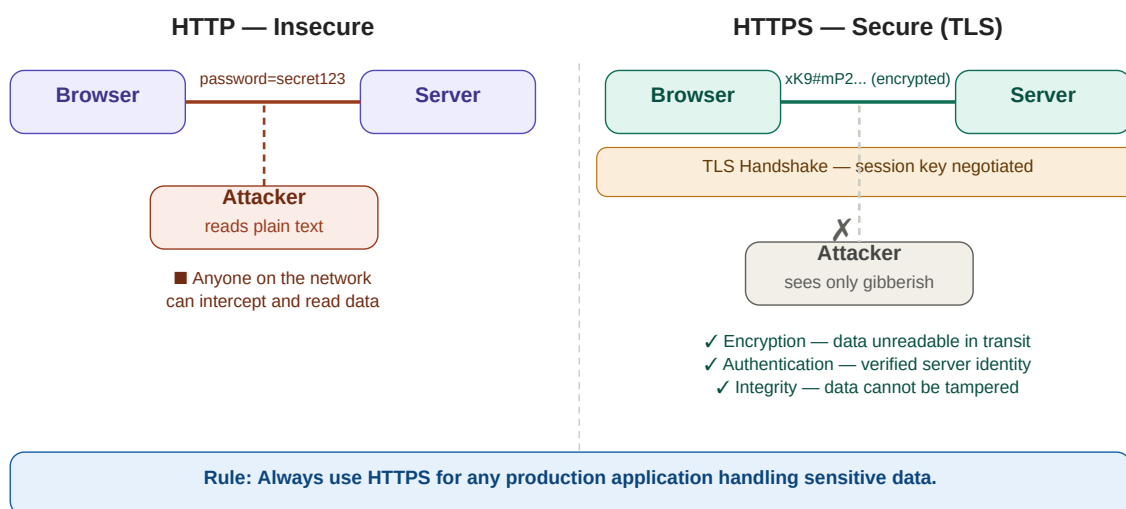
4. Application Layer Protocols

4.1 HTTP — Hypertext Transfer Protocol

Stateless request-response protocol for web pages and APIs over TCP (port 80). Client always initiates; server cannot push data unprompted. Each request is independent — no session memory.

4.2 HTTPS — HTTP Secure

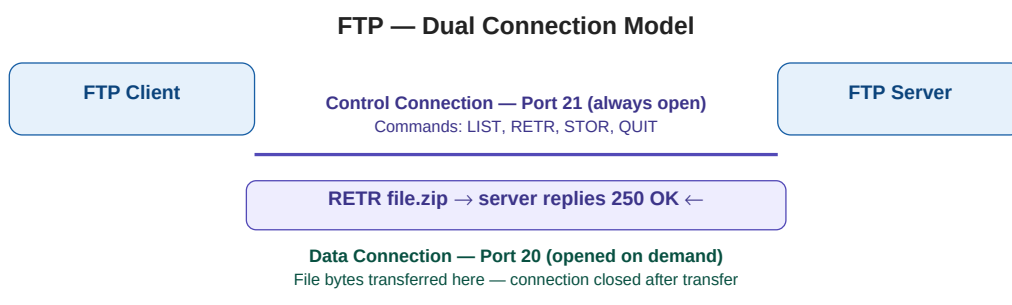
HTTP + TLS encryption (port 443). Adds **Encryption** (data unreadable in transit), **Authentication** (certificate verifies server identity), and **Integrity** (tamper detection).



HTTP — plain text interceptable vs HTTPS — TLS encrypted

4.3 FTP — File Transfer Protocol

Uses **two separate TCP connections**: Port 21 (control — stays open for commands) and Port 20 (data — opens per transfer, closes after). No encryption by default; use SFTP or FTPS instead.



Why two connections?

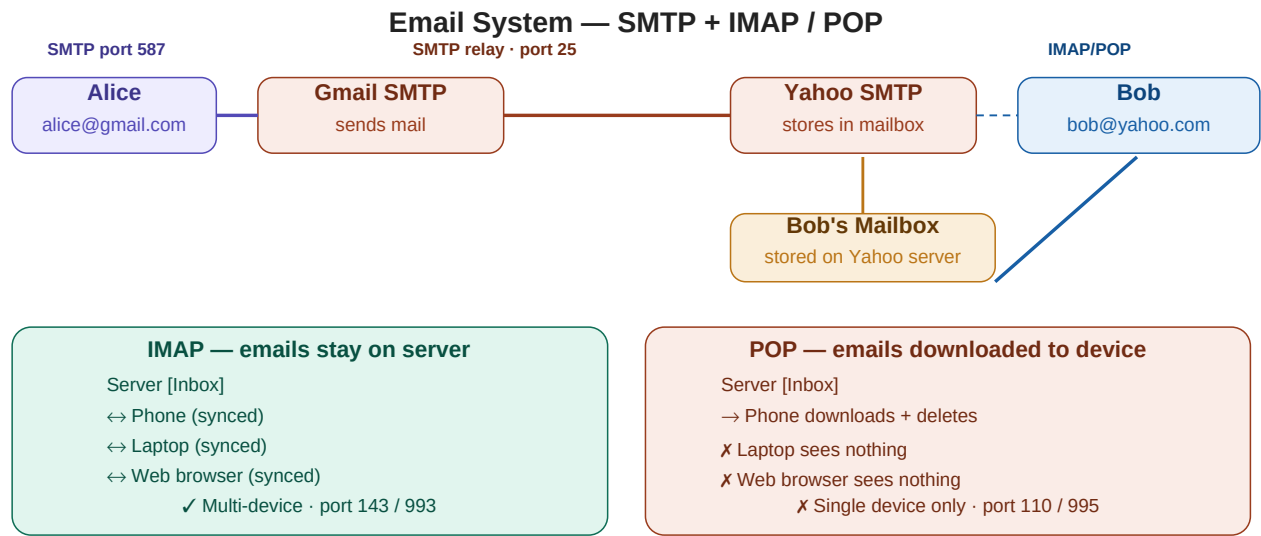
Control stays open the whole session — you can issue many commands without reconnecting.
Data opens only for the file transfer, then closes immediately — keeping control free.

FTP dual-connection — control (port 21) stays open, data (port 20) opens per file

4.4 SMTP — Simple Mail Transfer Protocol

Sends email only (port 587 client→server, port 25 server→server). Store-and-forward: if destination is unavailable, email is queued and retried. Cannot retrieve email — that requires IMAP or POP.

4.5 IMAP vs POP — Email Retrieval



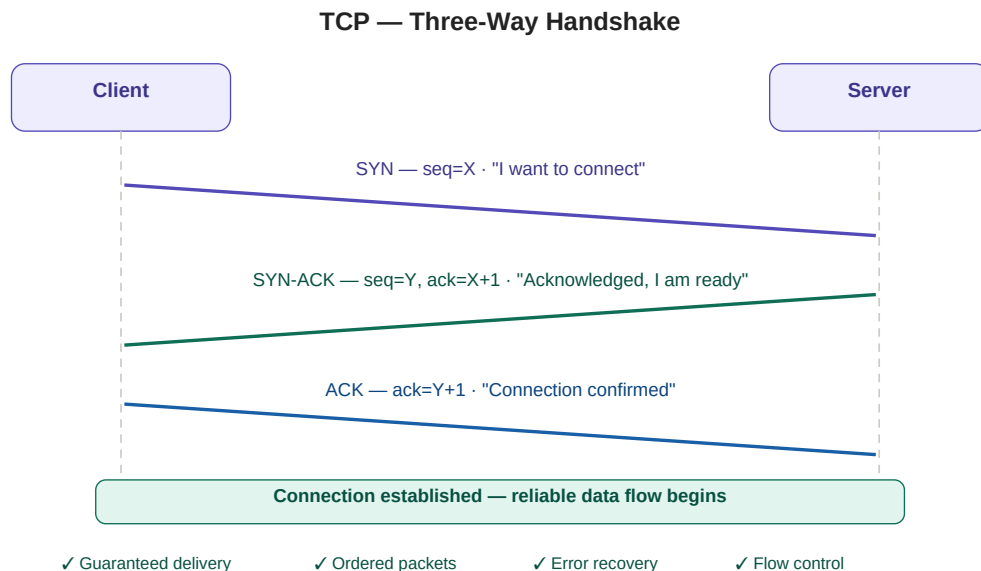
Email flow — SMTP sends, IMAP/POP retrieves, IMAP keeps server as master copy

Attribute	IMAP	POP
Email storage	Remains on server	Downloaded, deleted from server
Multi-device	✓ All devices synced	✗ Single device only
Offline	Limited (sync needed)	Full (all downloaded)
Port	143 / 993 (TLS)	110 / 995 (TLS)
Best for	Modern multi-device email	Legacy single-device

5. Transport Layer — TCP & UDP

5.1 TCP — Transmission Control Protocol

Connection-oriented, reliable, ordered. A three-way handshake establishes a connection before data flows. Every packet is acknowledged; lost packets retransmitted.



TCP three-way handshake — SYN → SYN-ACK → ACK before data flows

- **Delivery** — retransmits on timeout or 3 duplicate ACKs
- **Ordering** — packets numbered and reassembled in sequence
- **Flow control** — receiver advertises window size
- **Congestion control** — slow start + AIMD backoff

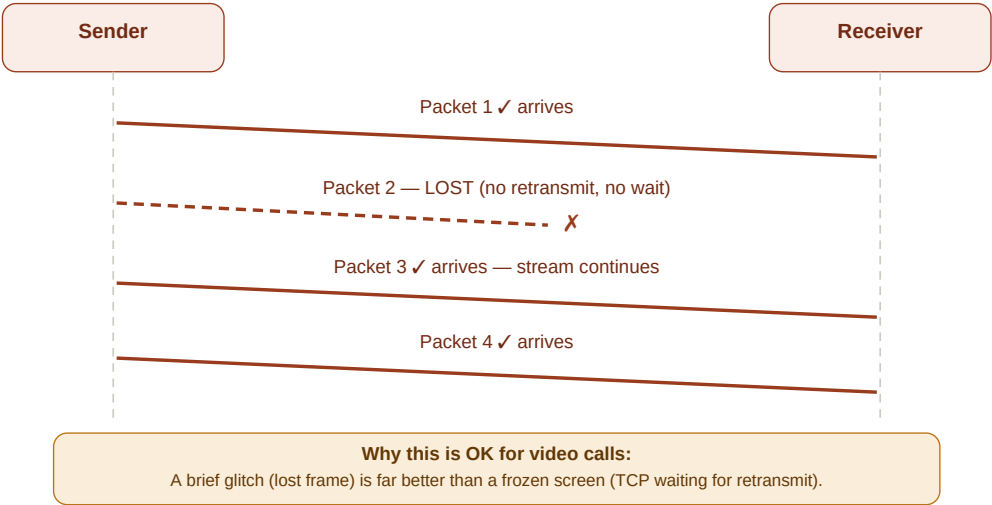
Overhead: 3-way handshake adds ~1.5 RTT before first byte. Head-of-line blocking: one lost packet stalls all subsequent packets.

Use TCP when data integrity is critical — messaging, file transfer, web, email.

5.2 UDP — User Datagram Protocol

Connectionless, unreliable, fast. No handshake, no ACKs, no retransmission. 8-byte header vs TCP's 20+ bytes. Data sent and forgotten.

UDP — Fire and Forget (No Connection)



UDP — no ACK, no retransmit, stream continues past lost packets

Why UDP for video calls: lost frame = brief glitch (tolerable). TCP retransmit = screen freeze (far worse).
Use UDP when latency is critical and minor loss is tolerable — video calls, streaming, gaming, DNS.

5.3 TCP vs UDP — Comparison

TCP vs UDP — Full Comparison

Attribute	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless — no setup
Reliability	Guaranteed delivery	No guarantee
Ordering	Guaranteed packet ordering	No ordering guarantee
Error recovery	Retransmits lost packets	No retransmission
Speed	Slower (overhead)	Faster (minimal overhead)
Header size	20+ bytes	8 bytes only
Use cases	Messaging, email, web, file transfer	Video calls, streaming, gaming, DNS
Choose when	Data integrity is critical	Latency is critical

TCP vs UDP across all key attributes

6. Protocol Selection by Application Type

The decision is always a tradeoff: **reliability vs latency**. Match the protocol to what the application can and cannot tolerate.

Protocol Selection by Application Type

Application	Model	Transport	Protocol	Why
Messaging (WhatsApp text)	Client-Server	TCP	HTTPS	Every message must arrive in order
Voice/Video calls (Zoom)	Hybrid	UDP	WebRTC	Latency > perfect delivery; glitch OK
Email — sending	Client-Server	TCP	SMTP	Store-and-forward, reliable delivery
Email — receiving	Client-Server	TCP	IMAP / POP	Must retrieve all emails correctly
Web browsing	Client-Server	TCP	HTTP/HTTPS	Full page must load intact
File download / upload	Client-Server	TCP	FTP / HTTPS	File must arrive byte-perfect
Live video streaming	Client-Server	UDP	RTMP/Custom	Real-time; minor loss tolerable

☒ TCP — reliability critical

☐ UDP — latency critical

Protocol selection — match each application to the right transport and app protocol

7. Security — HTTP vs HTTPS

HTTP sends data as plain text — a MITM attacker on the same network can read everything. HTTPS wraps HTTP in TLS, making intercepted data unreadable.

Always use HTTPS in production. HTTPS adds TLS beneath HTTP — semantics unchanged, transport encrypted.

End-to-end encryption (e.g. WhatsApp Signal Protocol) operates above TLS — even the server cannot read E2E messages.

8. Quick Revision Points

- Network protocol = rules for device communication (format, sequencing, error handling).
- OSI Layer 4 = TCP/UDP. Layer 7 = HTTP, SMTP, FTP, IMAP, POP.
- Client-Server is standard for messaging and web. WhatsApp is NOT P2P.
- P2P fails for messaging: no offline storage, NAT blocks connections, no group sync.
- HTTP is stateless request-response. HTTPS = HTTP + TLS on port 443.
- FTP: port 21 (control, always open) + port 20 (data, per-transfer).
- SMTP sends only. IMAP = server master copy, multi-device. POP = download to one device.
- TCP = reliable, ordered. Use for messaging, files, web, email.
- UDP = fast, no ACK. Use for video calls, streaming, gaming, DNS.

- Video loss tolerance: UDP glitch (brief flicker) beats TCP freeze (retransmit wait).
- Protocol choice = reliability vs latency based on what the application can tolerate.

9. Interview Questions & Key Terms

- 1. What is a network protocol and why is it needed?
- 2. OSI model — which layer does TCP operate at? Which layer does HTTP operate at?
- 3. Why do messaging apps use client-server instead of P2P? Name three specific reasons.
- 4. What is the difference between TCP and UDP? When would you use each?
- 5. Why is UDP preferred for video calls despite being unreliable?
- 6. What does FTP's dual-connection model solve?
- 7. What is the difference between SMTP, IMAP, and POP?
- 8. Why is IMAP preferred over POP for modern email clients?
- 9. Designing WhatsApp — which transport for text? Which for voice calls? Why?
- 10. What security risk does HTTP have? How does HTTPS solve it?
- 11. What happens when a packet is lost in TCP vs UDP?

Key Terms

Network Protocol	Rules defining how devices communicate (format, sequencing, error handling)
OSI Model	7-layer model — L4 Transport (TCP/UDP), L7 Application (HTTP/SMTP)
Client-Server	Central server handles all client requests and responses
P2P	Direct client-to-client without a server — fails for messaging
HTTP	Stateless request-response web protocol — Layer 7, port 80
HTTPS	HTTP + TLS encryption — port 443
FTP	File transfer — port 21 control, port 20 data
SMTP	Sends email between servers — outgoing only
IMAP	Retrieves email — keeps master copy on server (multi-device)
POP	Downloads email to one device, deletes from server
TCP	Reliable, ordered, connection-oriented transport — Layer 4
UDP	Fast, connectionless, unreliable transport — Layer 4
TLS/SSL	Cryptographic layer that encrypts HTTP into HTTPS
3-Way Handshake	TCP setup: SYN → SYN-ACK → ACK
Stateless	Each request is independent; server holds no memory between requests